

Assignment 9 - LDA, Logistic Regression, and Random Forest

Egla Mekonnen

Dataset Description

The dataset is from the OSMI (Open Sourcing Mental Illness) 2016 survey of mental health in the tech industry. It contains responses from 1,433 tech professionals worldwide regarding their mental health experiences, workplace environment, and demographic information.

- **Source:** Kaggle dataset: <https://www.kaggle.com/datasets/osmi/mental-health-in-tech-survey>
- **Purpose of the survey:** To investigate the likelihood of mental health issues in the tech industry, workplace policies, and personal attitudes toward mental health.
- **Observations and variables:** **Rows:** 1259 survey responses (each row = one respondent)
Columns: 27 variables, including demographics, employment characteristics, workplace attitudes, and personal/family mental health history.
- **Suitability for this project:** The dataset is well suited for binary classification because it has a yes/no target variable about whether someone has sought mental-health treatment, along with both categorical and numeric predictors. It also works well for inference methods like logistic regression and bootstrapping.

Shape of dataset: (1259, 27)

Research Question

"Can we predict whether a tech employee has sought mental health treatment based on workplace factors, demographics, and mental health attitudes?"

**** Predicting Treatment ****

Meaningful Predictors

- Age
- Gender
- Family history
- Work interference
- Remote work
- Tech company
- Benefits

- Care options
- Wellness program
- Leave
- Supervisor support
- observed consequences

Cleaning Data

- Dropping columns with too many missing value that have less importance to predicting Treatment

Handling Missing values

- Self_employed - only have 18 missing value , so we replace NaN with common value
- Work_interfere : is More complicated to fill missing value because the answers contains Never, Rarely, Sometime, Often and NaN are replaced unknown.

One-Hot encoding

```
df['work_interfere'] = df['work_interfere'].fillna('Unknown')
df = pd.get_dummies(df, columns=['work_interfere'], drop_first=True)
# one category is omitted from the resulting DataFrame. This omitted category becomes the
↳ reference category (or baseline). Work_inetreference_never is the baseline
```

```
# convert boolean to numeric values

bool_cols = ['work_interfere_Often', 'work_interfere_Rarely',
             'work_interfere_Sometimes', 'work_interfere_Unknown']

df[bool_cols] = df[bool_cols].astype(int)
```

Country column have multiple values so to turn this in to a numeric category

- normalize -> covert United States ==1 , other == 0

```
df = df.drop(columns=['comments', 'state', 'Timestamp', 'Country' ], errors='ignore')
```

- Handle inconsistent categorical responses

Age	0
Gender	0
self_employed	0
family_history	0
treatment	0
no_employees	0
remote_work	0
tech_company	0
benefits	0
care_options	0
wellness_program	0

```

seek_help            0
anonymity            0
leave               0
mental_health_consequence 0
phys_health_consequence 0
coworkers           0
supervisor          0
mental_health_interview 0
phys_health_interview 0
mental_vs_physical   0
obs_consequence      0
work_interfere_Often 0
work_interfere_Rarely 0
work_interfere_Sometimes 0
work_interfere_Unknown 0
Country_US          0
dtype: int64

```

Exploratory Data Analysis on clean data

Dataset shape: (1259, 27)

	Age	Gender	self_employed	family_history	treatment	remote_work	tech_company	benefits \
count	1.259000e+03	1259.000000	1259.000000	1259.000000	1259.000000	1259.000000	1259.000000	1259.000000
mean	7.942815e+07	0.806195	0.115965	0.390786	0.505957	0.298650	0.818904	0.702939
std	2.818299e+09	0.395435	0.320310	0.488121	0.500163	0.457848	0.385251	0.457145
min	-1.726000e+03	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	2.700000e+01	1.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000
50%	3.100000e+01	1.000000	0.000000	0.000000	1.000000	0.000000	1.000000	1.000000
75%	3.600000e+01	1.000000	0.000000	1.000000	1.000000	1.000000	1.000000	1.000000
max	1.000000e+11	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000

	care_options	wellness_program	seek_help	anonymity	work_interfere_Often	work_interfere_Rarely \
count	1259.000000	1259.000000	1259.000000	1259.000000	1259.000000	1259.000000
mean	0.352661	0.181890	0.198570	0.948372	0.114376	0.137411
std	0.477988	0.385907	0.399082	0.221363	0.318394	0.344417
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	1.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000	1.000000	0.000000	0.000000
75%	1.000000	0.000000	0.000000	1.000000	0.000000	0.000000
max	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000

	work_interfere_Sometimes	work_interfere_Unknown	Country_US
count	1259.000000	1259.000000	1259.000000
mean	0.369341	0.209690	0.596505
std	0.482818	0.407249	0.490793
min	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000
50%	0.000000	0.000000	1.000000
75%	1.000000	0.000000	1.000000
max	1.000000	1.000000	1.000000

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1259 entries, 0 to 1258
```

```
Data columns (total 27 columns):
```

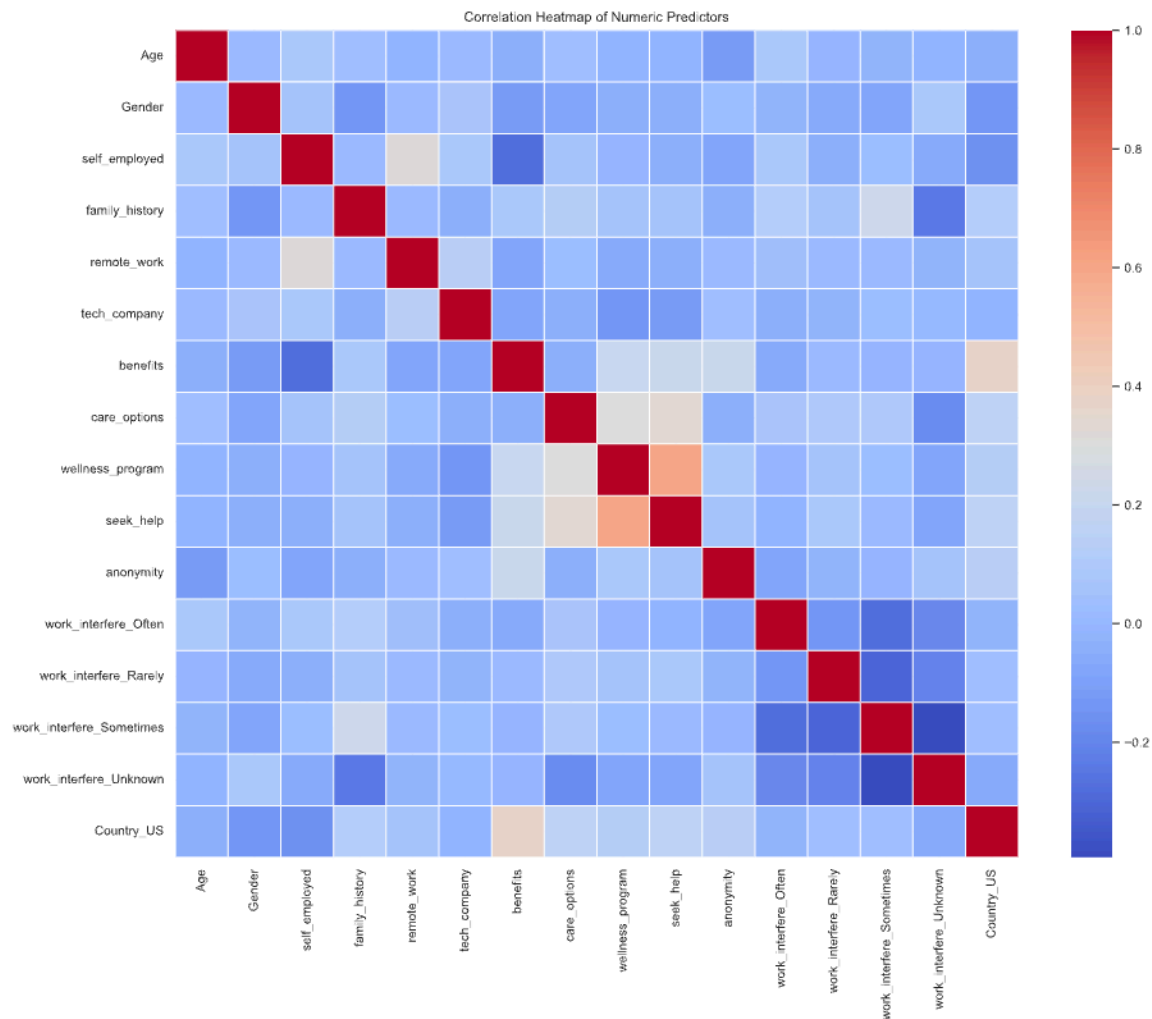
#	Column	Non-Null Count	Dtype
0	Age	1259 non-null	int64
1	Gender	1259 non-null	int64
2	self_employed	1259 non-null	int64
3	family_history	1259 non-null	int64
4	treatment	1259 non-null	int64
5	no_employees	1259 non-null	object
6	remote_work	1259 non-null	int64
7	tech_company	1259 non-null	int64
8	benefits	1259 non-null	float64
9	care_options	1259 non-null	float64
10	wellness_program	1259 non-null	float64
11	seek_help	1259 non-null	float64
12	anonymity	1259 non-null	float64
13	leave	1259 non-null	object
14	mental_health_consequence	1259 non-null	object
15	phys_health_consequence	1259 non-null	object
16	coworkers	1259 non-null	object
17	supervisor	1259 non-null	object
18	mental_health_interview	1259 non-null	object
19	phys_health_interview	1259 non-null	object

```

20 mental_vs_physical      1259 non-null  object
21 obs_consequence         1259 non-null  object
22 work_interfere_Often     1259 non-null  int64
23 work_interfere_Rarely    1259 non-null  int64
24 work_interfere_Sometimes 1259 non-null  int64
25 work_interfere_Unknown   1259 non-null  int64
26 Country_US              1259 non-null  int64
dtypes: float64(5), int64(12), object(10)
memory usage: 265.7+ KB
None

```

Correlation between predictors

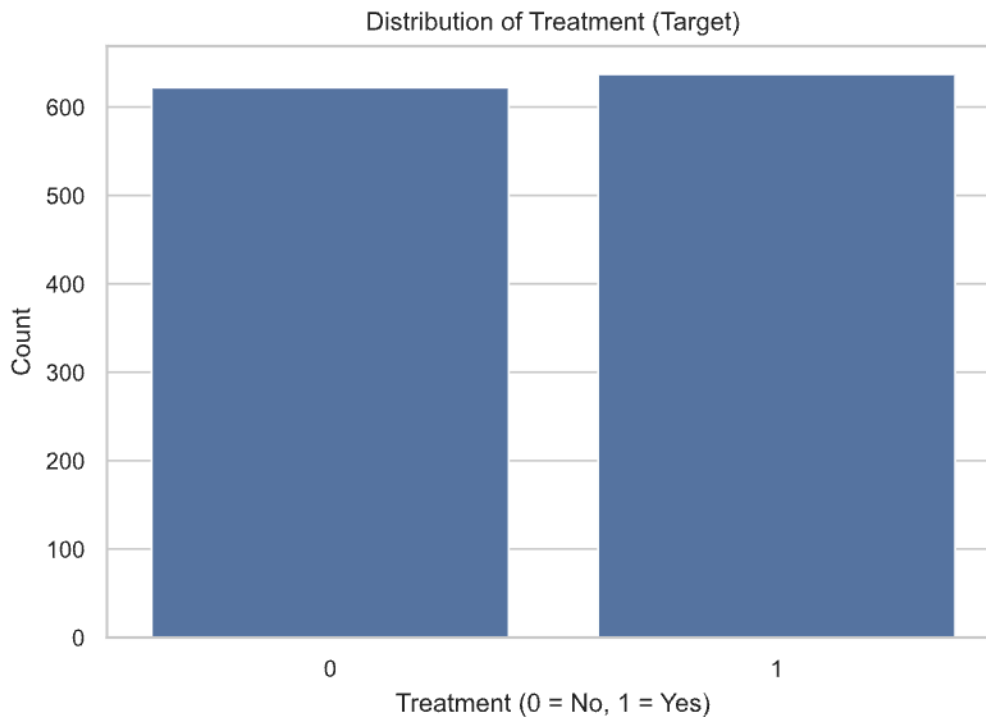


Moderate Correction Between predictors

- wellness_program and seek_help ≈ 0.596
- wellness_program and care_options ≈ 0.307
- seek_help and care_options ≈ 0.332

Features and Targets

Target is Treatment (1 = Received Treatment, 0= did not receive Treatment)



```
treatment
1    50.595711
0    49.404289
Name: proportion, dtype: float64
```

This shows a balanced dataset, the two classes are close to each other

```
treatment
1    50.595711
0    49.404289
```

Relationship between Features and Target Variables

Interpretation of Plot(after plotting each relationship)

- Gender has a weak relationship with treatment.
- Being self-employed does **not strongly affect** seeking mental health treatment. (Weak predictor.)
- Family history is a **strong predictor**.
- People with family history of mental illness are much more likely to seek treatment.
- Remote work has **little to no effect** on treatment. (Matches low correlation.)
- Working in a tech company does **not strongly predict** treatment.
- Country_US is a **moderate predictor** of treatment

Splitting data , Train-test split

```
# Target column
y = df['treatment'] # your target is 'treatment'

# Features: select all numeric columns except the target
X = df.select_dtypes(include='number').drop(columns=['treatment'])
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Scaling

Building Logistic regression

```
from sklearn.linear_model import LogisticRegression

log_reg = LogisticRegression(random_state=42)
log_reg.fit(X_train_scaled, y_train)
```

```
LogisticRegression(random_state=42)
```

```
# Predicted class labels
y_pred_log = log_reg.predict(X_test_scaled)

# Predicted probabilities for positive class (treatment = 1)
y_prob_log = log_reg.predict_proba(X_test_scaled)[: , 1]
```

Model Evaluation

Confusion Matrix:

```
[[ 95  29]
 [ 22 106]]
```

Accuracy

Logistic regression coefficients

Coefficients:

	feature	coefficient
0	intercept	-0.372973
1	Age	-0.073181
2	Gender	-0.269252
3	self_employed	-0.053601
4	family_history	0.549624
5	remote_work	-0.009861
6	tech_company	-0.023683
7	benefits	0.185140
8	care_options	0.520762
9	wellness_program	-0.005938
10	seek_help	-0.045282
11	anonymity	0.058802
12	work_interfere_Often	1.054456
13	work_interfere_Rarely	0.786989
14	work_interfere_Sometimes	1.282303
15	work_interfere_Unknown	-1.002722
16	Country_US	-0.036433

Interpretation of Coefficients

"Positive coefficients indicate predictors that increase the likelihood of seeking treatment, while negative coefficients reduce that likelihood. Coefficients near zero have minimal impact."

Work_interference and Family history have the highest positive coefficient

when we interpret the coefficient of Work_Interference because we have 4 dummy variables.

The baseline for this column is Work_interfere_Never, Each dummy coefficient shows how that category compares to the baseline ("Never").

- work_interfere_Sometimes = 1.282303, Strongest positive
Respondents whose mental health *sometimes* interferes with work are **much more likely** to seek treatment compared to those who report **no interference**. same as work_interfere_Often = 1.054456
- work_interfere_Unknown = -1.00
Respondents who skipped or refused to answer the question ("Unknown") are much **less** likely to have sought treatment.

LDA

```
LinearDiscriminantAnalysis()
```

LDA Accuracy: 0.8333333333333334

Confusion Matrix (LDA):

```
[[ 91  33]
 [  9 119]]
```

Classification Reports (Logistic vs LDA)

Logistic Regression:

	precision	recall	f1-score	support
No Treatment	0.81	0.77	0.79	124
Treatment	0.79	0.83	0.81	128
accuracy			0.80	252
macro avg	0.80	0.80	0.80	252
weighted avg	0.80	0.80	0.80	252

LDA:

	precision	recall	f1-score	support
No Treatment	0.91	0.73	0.81	124
Treatment	0.78	0.93	0.85	128
accuracy			0.83	252
macro avg	0.85	0.83	0.83	252
weighted avg	0.85	0.83	0.83	252

	model	test_accuracy	test_auc
0	Logistic Regression	0.797619	0.883128
1	LDA	0.833333	0.885585

Bootstrap analysis for the two models

```
# bootstrap confidence interval
alpha = 0.05
ci_low, ci_high = np.percentile(diffs, [100*alpha/2, 100*(1 - alpha/2)])

print("Mean accuracy difference (Logistic - LDA):", np.mean(diffs))
```

```
Mean accuracy difference (Logistic - LDA): -0.21738633013001898
```

```
print("95% CI for the accuracy difference:", (ci_low, ci_high))
```

```
95% CI for the accuracy difference: (np.float64(-0.36375951019673164), np.float64(0.007473642521577606))
```

NOTE:

The bootstrap results show that LDA generally performs better than Logistic Regression. On average, LDA's accuracy is about 20% higher. The 95% confidence interval (-0.358 to 0.008) is mostly below zero, meaning LDA almost always outperforms Logistic Regression. Logistic Regression may occasionally perform similarly, but overall LDA is clearly the more reliable model.

Random Forest Classification

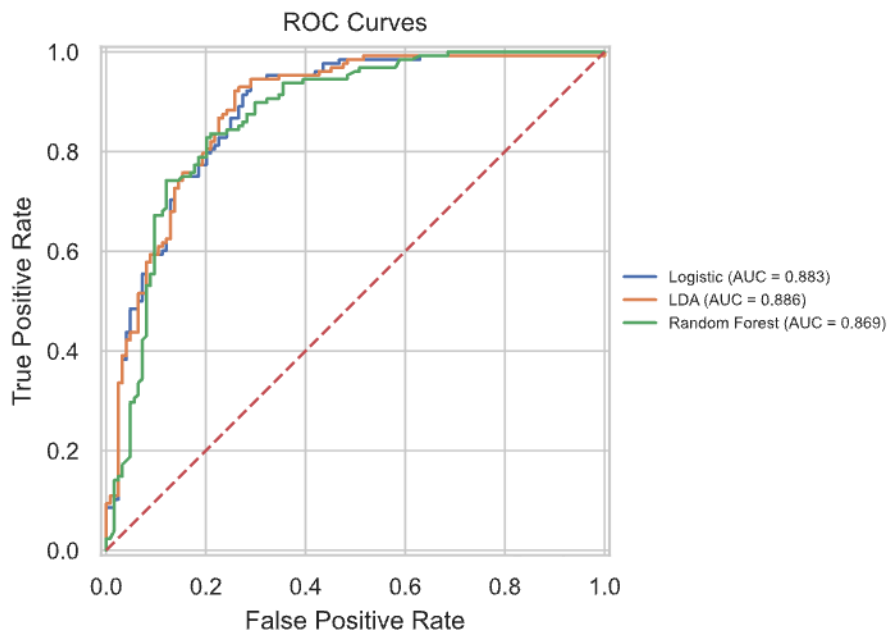
```
[<matplotlib.lines.Line2D object at 0x157007be0>]
```

```
<matplotlib.legend.Legend object at 0x1570166a0>
```

```
Text(0.5, 0, 'False Positive Rate')
```

```
Text(0, 0.5, 'True Positive Rate')
```

```
Text(0.5, 1.0, 'ROC Curves')
```



	model	test_accuracy	test_auc
0	Logistic Regression	0.797619	0.883128
1	LDA	0.833333	0.885585
2	Random Forest	0.805556	0.869298

Across all three models, Logistic Regression, LDA, and Random Forest—performance is fairly strong, with accuracy between **0.79 and 0.83** and AUC between **0.86 and 0.89**, indicating that all models are capable of distinguishing between employees who sought mental-health treatment and those who did not.

1. Linear Discriminant Analysis (LDA)

- Best overall accuracy: 0.833
- Highest AUC: 0.886
- LDA performs the strongest of the three models, suggesting that the predictors in this dataset separate the two classes (treatment vs. no treatment) fairly well under linear boundaries. The model benefits from the balanced classes and the presence of multiple binary predictors.

2. Logistic Regression

- Accuracy: 0.798
- AUC: 0.883 (very close to LDA)
- Logistic Regression performs similarly well in ranking predictions (AUC), but slightly worse in accuracy. This suggests that although Logistic Regression can order individuals correctly by risk, its actual classification threshold cuts slightly less effectively than LDA.

3. Random Forest

- Accuracy: 0.806

- AUC: 0.869
- Random Forest performs reasonably well, but both linear models do better. Since most predictors are simple binary variables, there may not be enough complex interactions for Random Forest to take advantage of.

LDA is the strongest model overall, showing the highest accuracy and AUC. Logistic Regression follows closely, especially in AUC, meaning it still captures the underlying relationship well. Random Forest performs acceptably but does not exceed the simpler linear models, suggesting that in this dataset, linear relationships and class separation are more important than complex nonlinear patterns